

Practice Assignment – 1 (Experiment 5)

Object Oriented Programming using Java

Instructions

- This assignment covers concepts from **Experiment 1 to Experiment 4**.
- Students must solve **4 questions** based on their roll number.
- Use proper indentation, comments, and meaningful variable names.

Question Allocation Based on Roll Number

Last Digit of Roll No	Questions to Solve
1	1, 11, 21, 31
2	2, 12, 22, 32
3	3, 13, 23, 33
4	4, 14, 24, 34
5	5, 15, 25, 35
6	6, 16, 26, 36
7	7, 17, 27, 37
8	8, 18, 28, 38
9	9, 19, 29, 39
0	10, 20, 30, 40

Assignment Questions

Section A: Basics of Java and Control Structures (Exp-1)

1. Write a Java program to check whether a given number is positive, negative, or zero.
2. Write a Java program to find the factorial of a number using a **while** loop.
3. Write a Java program to reverse a given integer number.
4. Write a Java program to check whether a given number is a palindrome.

5. Write a Java program to calculate the sum of digits of a number.
6. Write a Java program to print multiplication table of a number using **for** loop.
7. Write a Java program to check whether a given number is an **Armstrong number** or not. The number should be accepted using **Scanner** and logic must use a loop.
8. Write a Java program to count the total number of **vowels and consonants** in a given string. Use conditional statements and loops.
9. Write a Java program to implement a **menu-driven calculator** using **switch--case** for addition, subtraction, multiplication, and division.
10. Write a Java program to count the number of digits in a given number.

Section B: Classes, Objects and Methods (Exp–2)

11. Create a class **Student** with methods to:
 - accept marks of N subjects using **Scanner**
 - calculate total and average marks
 - display grade using **if--else**
12. Create a class **Employee** with methods to:
 - accept employee details (name, Id, Salary) using command line arguments.
 - Find the hightest and lowest salary of all employee.
 - display details of the employee with higher salary and lower salary.
13. Create a class **Point** with methods to:
 - accept coordinates of two points
 - calculate distance between them using formula
14. Create a class **ArrayOperation** with methods to:
 - accept N integers (min 3, max 10)
 - find maximum and minimum values
15. Create a class **Matrix** with methods to:
 - accept two matrices
 - perform matrix addition
 - display the resultant matrix

16. Create a class **Book** with properties title, author, and year. Implement methods to:

- display book details
- sort all books based on properties.

17. Create a class **NumberAnalysis** with methods to:

- check whether a number is **Oddish or Evenish**
- display appropriate message

A number is Oddish if the sum of all of its digits is odd, and a number is Evenish if the sum of all of its digits is even.

18. Create a class **DigitPattern** with methods to:

- accept a number(N)
- print a digit-based pattern using loops

Ex: N = 4

Pattern

```
1
2 2
3 3 3
4 4 4 4
```

19. Create a class **NumberSystem** with methods to convert:

- Binary to Decimal
- Binary to Hexadecimal
- Decimal to Hexadecimal
- Hexadecimal to Binary

20. Create a class **UniqueNumber** with methods to:

- accept N numbers
- identify all unique three-digit numbers
- count how many such numbers exist

Section C: Constructors and this Keyword (Exp-3)

21. Create a class using a constructor to check whether a given number is a **Disarium number**. A number is said to be Disarium if the sum of its digits raised to their respective positions is the number itself.

Ex: 135

$$1^1 + 3^2 + 5^3 = 1 + 9 + 125 = 135$$

22. Create a class using constructor and **this** keyword to check whether a number is a **Harshad number**. A number is said to be the Harshad number if it is divisible by the sum of its digits. For example, if the number is 156, then the sum of its digits will be $1 + 5 + 6 = 12$. Since 156 is divisible by 12. So, 156 is a Harshad number.
23. Create a class using constructor to check whether a given number is an **Auto-morphic number**. A number is called an automorphic number if and only if the square of the given number ends with the same number itself. For example, 25, 76 are automorphic numbers because their square is 625 and 5776, respectively.
24. Create a class to calculate area and circumference of circle using constructor overloading.
25. Write a Java program to calculate simple and compound interest using constructor overloading.
26. Create a class using constructor to check whether a given number is a **Perfect number**. A perfect number is a number that can be written as the sum of its factors, excluding the number itself. Ex: 28 is a perfect number, since $1 + 2 + 4 + 7 + 14 = 28$
27. Create a class using constructor to convert a **decimal number to binary** without using built-in conversion methods.
28. Create a class using constructor and **this** keyword to convert a **Hexadecimal number to decimal** without using built-in conversion methods.
29. Create a class using constructor to generate and display the first N terms of a number series where N is passed as a parameter.
30. Create a class using constructor to accept an array of integers and count how many elements are **three-digit numbers**.

Section D: Static, Object Lifecycle and HAS-A Relationship (Exp-4)

31. Create a utility class with:
 - a static method to calculate GCD of two numbers
 - a non-static method to calculate LCM using GCD
32. Write a Java program to demonstrate **object lifecycle** by:
 - creating multiple references to the same object
 - making the object eligible for garbage collection

33. Create a class that uses a static variable to count how many objects of that class are created.
34. Create a HAS-A relationship between classes **Student** and **Address** and display complete student information, where multiple students may share the same address object.
35. Write a Java program to demonstrate a HAS-A relationship between **Employee** and **Department**, where department details are passed to employee objects through constructor.
36. Write a Java program to store details of N students ($3 \leq N \leq 10$) using an array of objects and display the topper based on total marks.
37. Write a Java program where a static method determines whether a number is prime and a non-static method generates all prime numbers up to a given limit.
38. Write a Java program to demonstrate passing an object as an argument to a method, modifying its data members inside the method, and reflecting the changes in the calling method.
39. Develop a Java class to represent information about software company employees with information like salary code, designation, salary, department. Implement method to display the employee details
40. The daily maximum temperature recorded in **10 cities** during one week (July 1 to July 7) has been tabulated as shown below.

City / Day	1	2	3	4	5	6	7
City 1							
City 2							
City 3							
⋮							
City 10							

Write a Java program to perform the following tasks:

- Read the temperature values into a two-dimensional array `temperature[10][7]`.
- For each city, determine the **highest temperature recorded** and display the corresponding **day** on which it occurred.
- For each day, determine the **highest temperature recorded** and display the corresponding **city** in which it occurred.

- Determine the **overall highest temperature** recorded during the week and display:
 - the value of the temperature
 - the day on which it occurred
 - the city in which it was recorded

Outcome

After completing this assignment, students will be able to:

- Apply Java basics with control structures.
- Use classes, objects, methods, and constructors effectively.
- Implement logic-based programs using Java.
- Understand object lifecycle and object relationships.